Lecture 21: Sharding (Part 3)

Data Availability

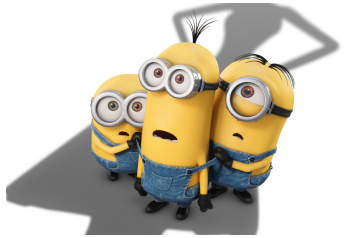
communication scaling
coded Merkle tree

Data Availability Attacks in Blockchain Systems

and How to Solve It.

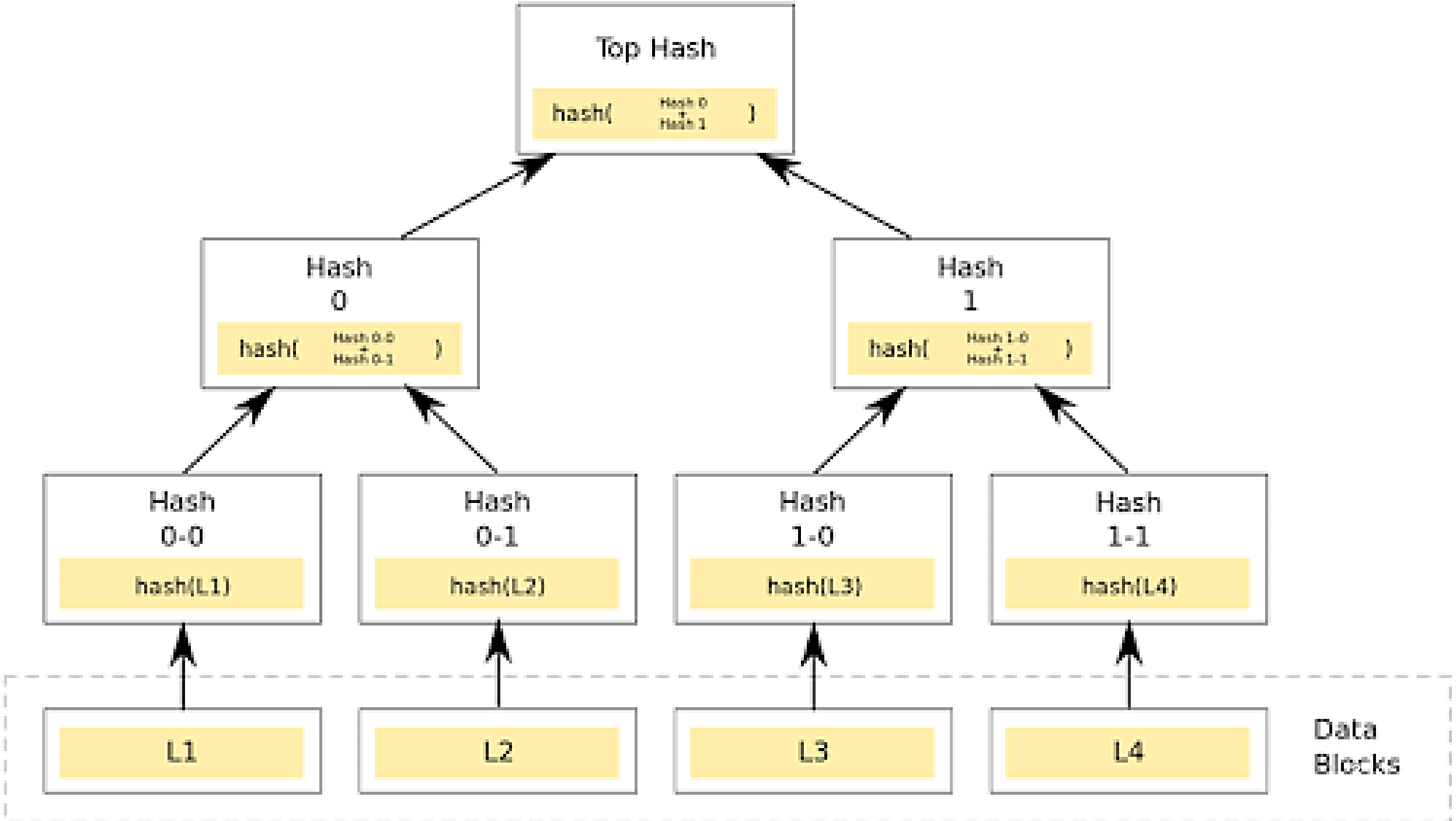
I have all the data you want

Really …



Merkle Tree

- A hash tree, also known as a Merkle tree, is a tree in which each leaf node is labeled with the cryptographic hash of a data block, and each non-leaf node is labeled with the cryptographic hash of its child nodes' labels. The majority of hash tree implementations are binary (each node has two child nodes), but they can also have many more child nodes.


Top Hash
hash(Hash 0 + Hash 1)
Hash 0
hash(Hash 0-0 + Hash 0-1)
Hash 1
hash(Hash 1-0 + Hash 1-1)
Hash 0-0
hash(L1)
Hash 0-1
hash(L2)
Hash 1-0
hash(L3)
Hash 1-1
hash(L4)
L1
L2
L3
L4
Data Blocks

- Merkle trees, also known as Binary hash trees, are a prevalent sort of data structure in computer science.

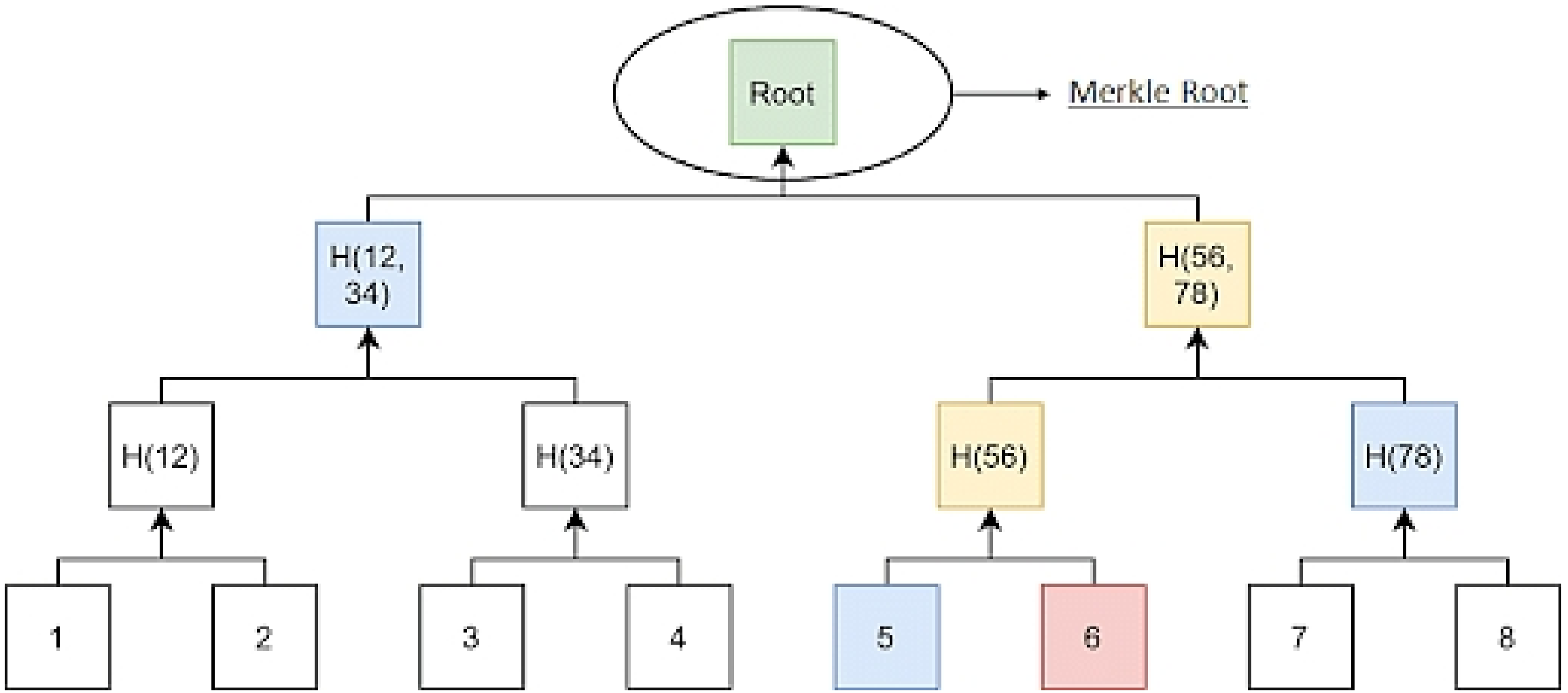
- In bitcoin and other cryptocurrencies, they're used to encrypt blockchain data more efficiently and securely.

- It's a mathematical data structure made up of hashes of various data blocks that summarize all the transactions in a block.

- It also enables quick and secure content verification across big datasets and verifies the consistency and content of the data.

What Is a Merkle Root?

- A Merkle root is a simple mathematical method for confirming the facts on a Merkle tree.
- They're used in cryptocurrency to ensure that data blocks sent through a peer-to-peer network are whole, undamaged, and unaltered.
- They play a very crucial role in the computation required to keep cryptocurrencies like bitcoin and ether running.

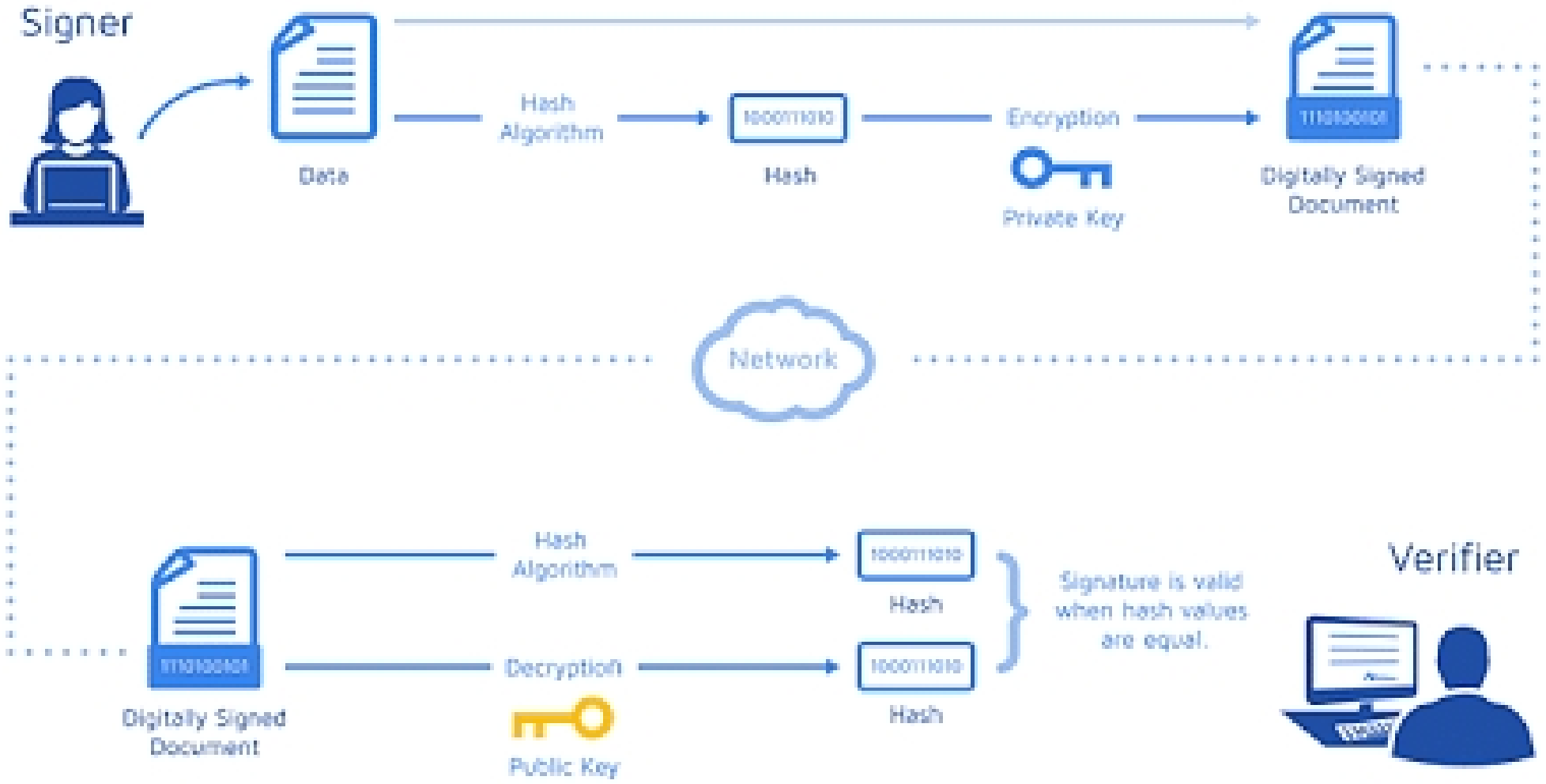

Root
Merkle Root
H(12, 34)
H(56, 78)
H(12)
H(34)
H(56)
H(78)
1
2
3
4
5
6
7
8

Cryptographic Hash Functions

- A hash function maps any type of arbitrary data of any length to a fixed-size output. It is commonly used in cryptography since it is a cryptographic function.
- They are efficient and are well-known for one property: they are irreversible. It's a one-way function that's only meant to work in one direction.
- Some of the Hash families available are Message Direct (MD), Secure Hash Function (SHF), and RIPE Message Direct (RIPEMD).
- Now, take an example, if you use the SHA256 hash algorithm and pass 101Blockchains as input, you will get the following output
- fbffd63a60374a31aa9811cbc80b577e23925a5874e86a17f712bab874f33ac9

Working of Merkle Trees

- A Merkle tree totals all transactions in a block and generates a digital fingerprint of the entire set of operations, allowing the user to verify whether it includes a transaction in the block.
- Merkle trees are made by hashing pairs of nodes repeatedly until only one hash remains; this hash is known as the Merkle Root or the Root Hash.
- They're built from the bottom, using Transaction IDs, which are hashes of individual transactions.
- Each non-leaf node is a hash of its previous hash, and every leaf node is a hash of transactional data.


Signer
Data
Hash Algorithm
Hash
Encryption
Private Key
Digitally Signed Document
Network
Digitally Signed Document
Hash Algorithm
Decryption
Public Key
Hash
Hash
Signature is valid when hash values are equal.
Verifier

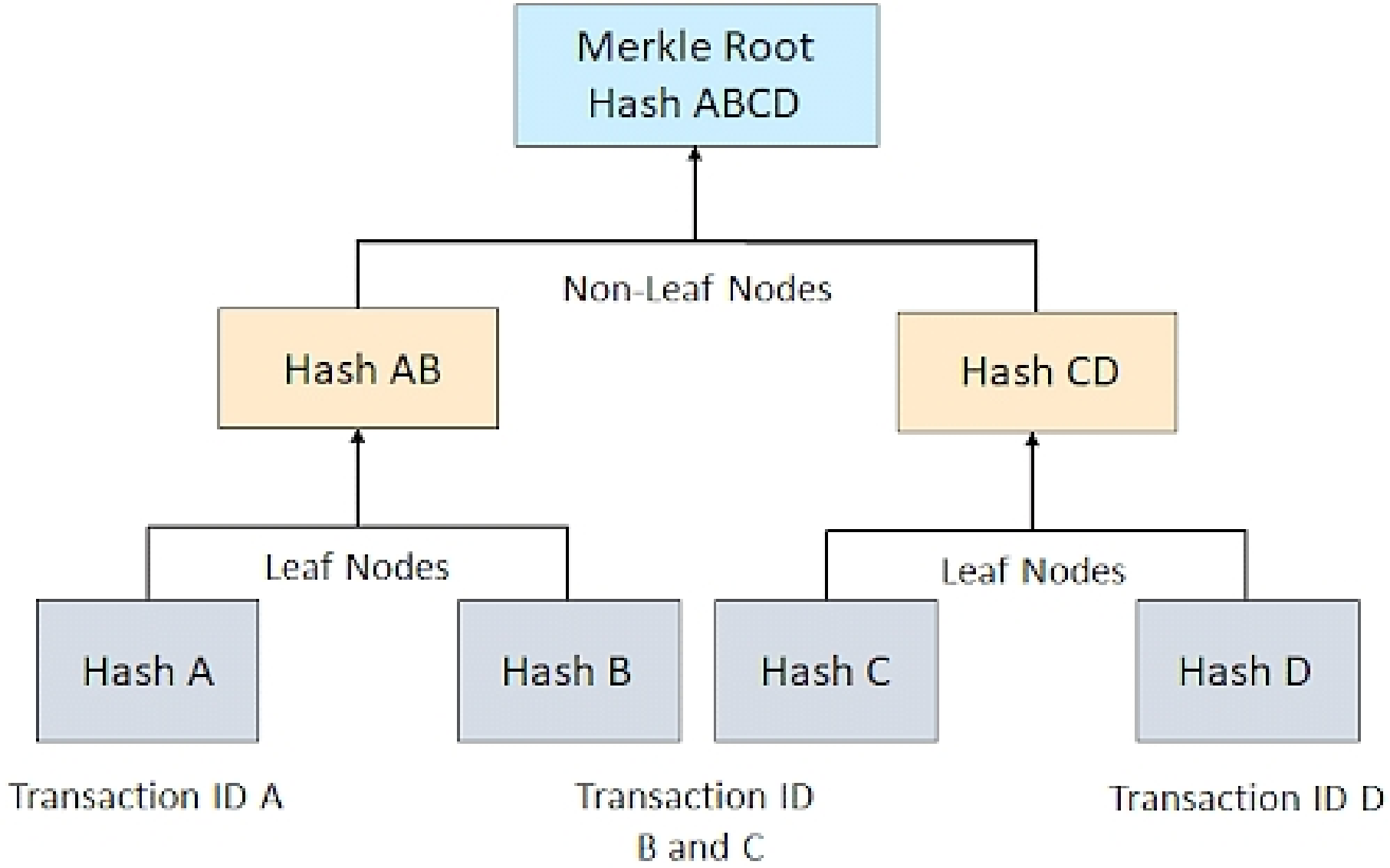
Now, look at a little example of a Merkle Tree in Blockchain to help you understand the concept.

Consider the following scenario: A, B, C, and D are four transactions, all executed on the same block. Each transaction is then hashed, leaving you with:

- Hash A
- Hash B
- Hash C
- Hash D

The hashes are paired together, resulting in:
Hash AB
and
Hash CD
And therefore, your Merkle Root is formed by combining these two hashes: Hash ABCD.

In reality, a Merkle Tree is much more complicated (especially when each transaction ID is 64 characters long). Still, this example helps you have a good overview of how the algorithms work and why they are so effective.


Merkle Root
Hash ABCD
Non-Leaf Nodes
Hash AB
Hash CD
Leaf Nodes
Leaf Nodes
Hash A
Hash B
Hash C
Hash D
Transaction ID A
Transaction ID
B and C
Transaction ID D

Benefits of Merkle Tree in Blockchain

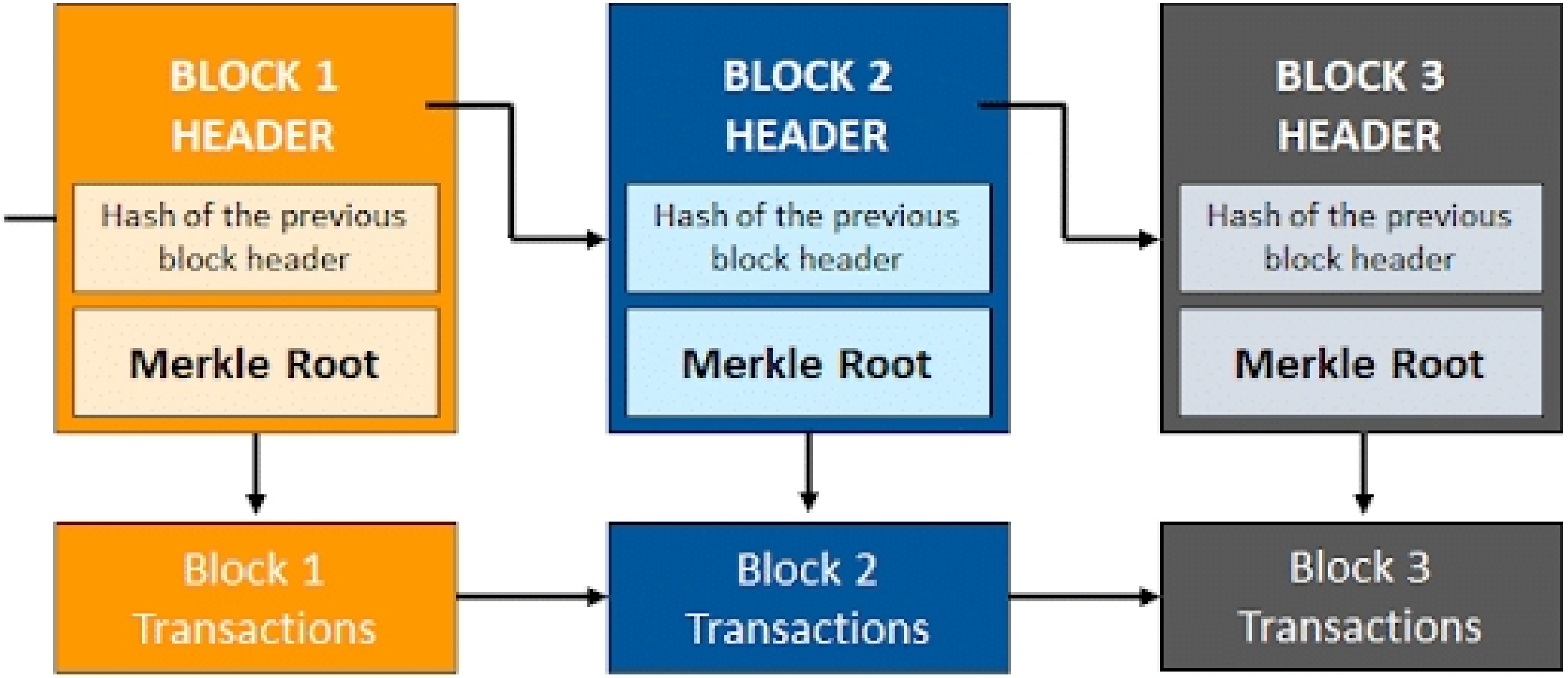
Merkle trees provide four significant advantages -

- Validate the data's integrity: It can be used to validate the data's integrity effectively.
- Takes little disk space: Compared to other data structures, the Merkle tree takes up very little disk space.
- Tiny information across networks: Merkle trees can be broken down into small pieces of data for verification.
- Efficient Verification: The data format is efficient, and verifying the data's integrity takes only a few moments.

Why Is It Essential to Blockchain?

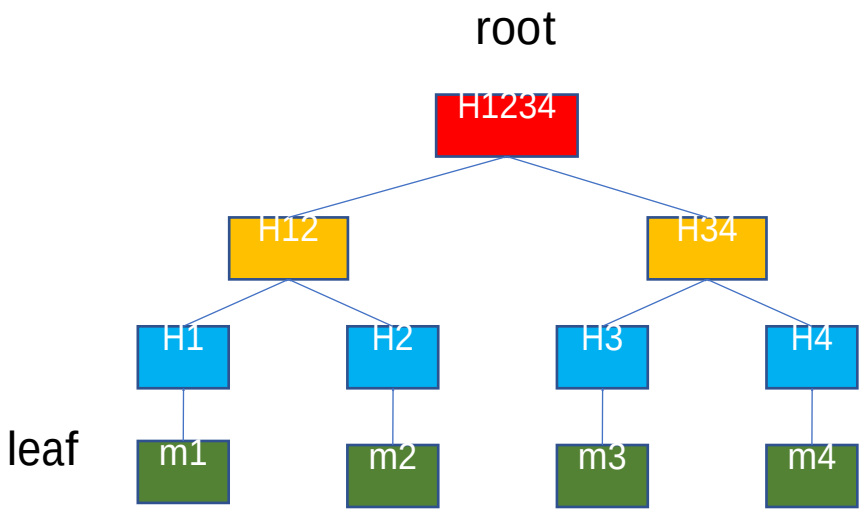
- Think of a blockchain without Merkle Trees to get a sense of how vital they are for blockchain technology. Let’s have one of Bitcoin because its use of Merkle Trees is essential for the cryptocurrency and easier to grasp.
- If Bitcoin didn't include Merkle Trees, per se, every node on the network would have to retain a complete copy of every single Bitcoin transaction ever made. One can imagine how much information that would be.
- Any authentication request on Bitcoin would require an enormous amount of data to be transferred over the network: therefore, you'll need to validate the data on your own.
- To confirm that there were no modifications, a computer used for validation would need a lot of computing power to compare ledgers.

- Merkle Trees are a solution to this issue. They hash records in accounting, thereby separating the proof of data from the data itself.
- Proving that giving tiny amounts of information across the network is all that is required for a transaction to be valid.
- Furthermore, it enables you to demonstrate that both ledger variations are identical in terms of nominal computer power and network bandwidth.



Merkle Tree breaking the data into tiny parts of information

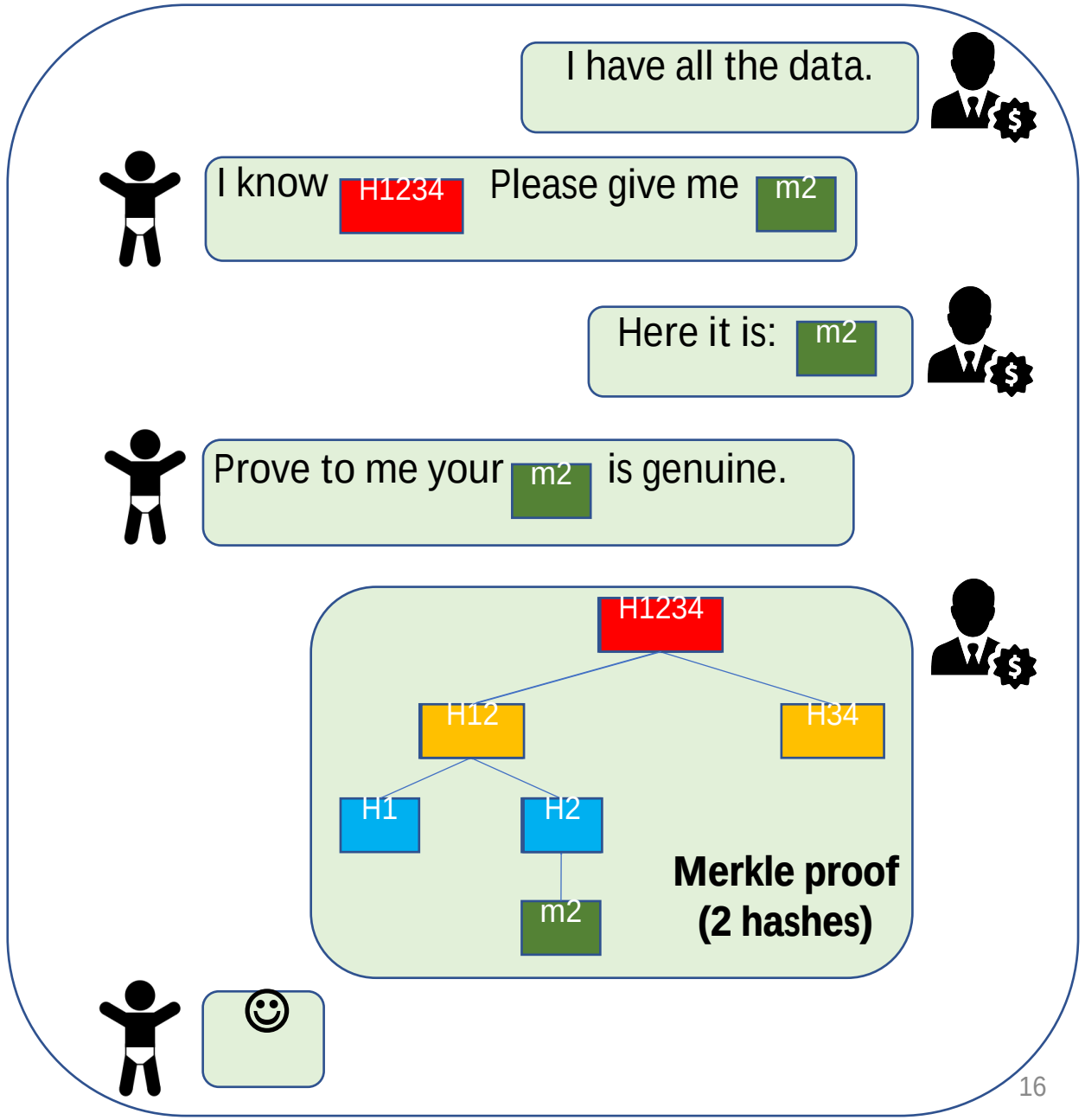
Merkle Tree Recap



A Merkle proof is a succinct proof of that a message is one of the leaves of a Merkle tree.

Tamper-free

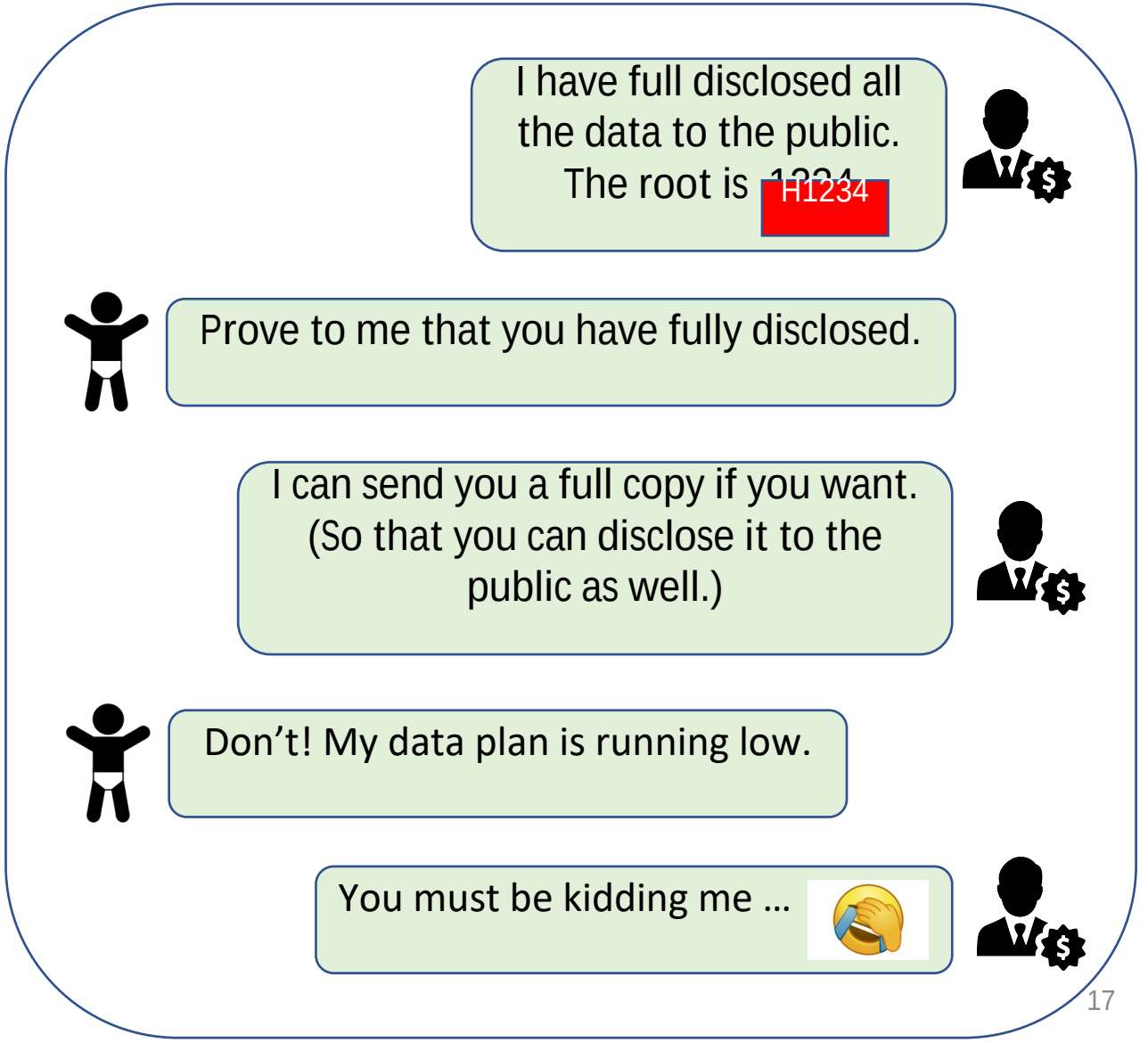
As long as you have the root, no one can deceive you to accept fake massages.



But There Are Other Problems …

How can you check the availability of all the messages without fully downloading them?

Does this problem matter?

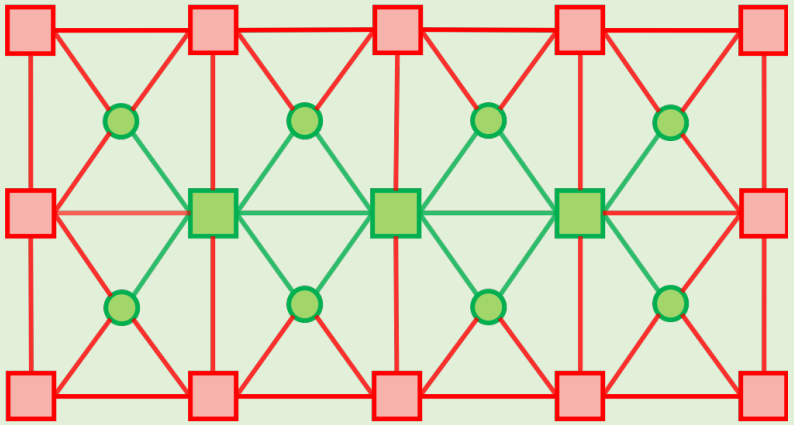


- **Motivation: blockchain light node**
- The data availability problem: assumptions and definition
- How could erasure coding help?
- The optimal solution
- Performance

Play motivating video

- Motivation: blockchain light node
- **The data availability problem: assumptions and definition**
- How could erasure coding help?
- The optimal solution
- Performance

Assumptions



- **No orphan**: Every node is connected to at least 1 honest full node
- **Every honest node gossips/re-broadcast**
- **Dishonest majority**

The data availability problem [1]

When

I published both Body & H

I received everything.

Body is good, safe to accept H

Then

I only have H

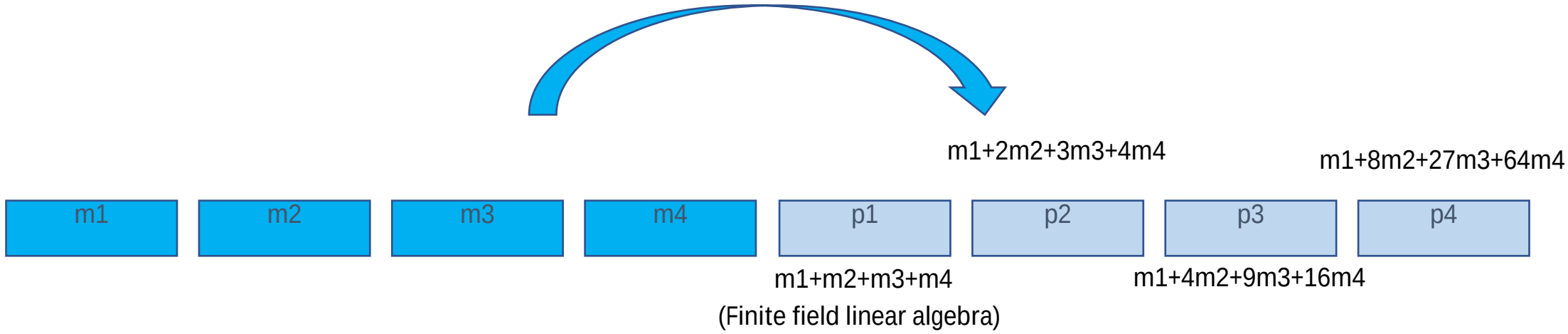
How to check whether Body is full available in the system or not?

Sampling is the only rescue.
How to make it efficient?

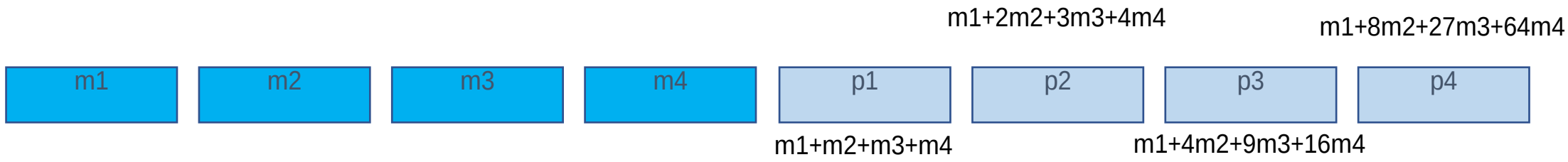
[1] Al-Bassam, M., Sonnino, A., Buterin, V., *Fraud and data availability proofs: Maximising light client security and scaling blockchains with dishonest majorities, 2018*

- Motivation: blockchain light node
- The data availability problem: definition and assumptions
- **How could erasure coding help?**
- The optimal solution
- Performance

Erasure Coding



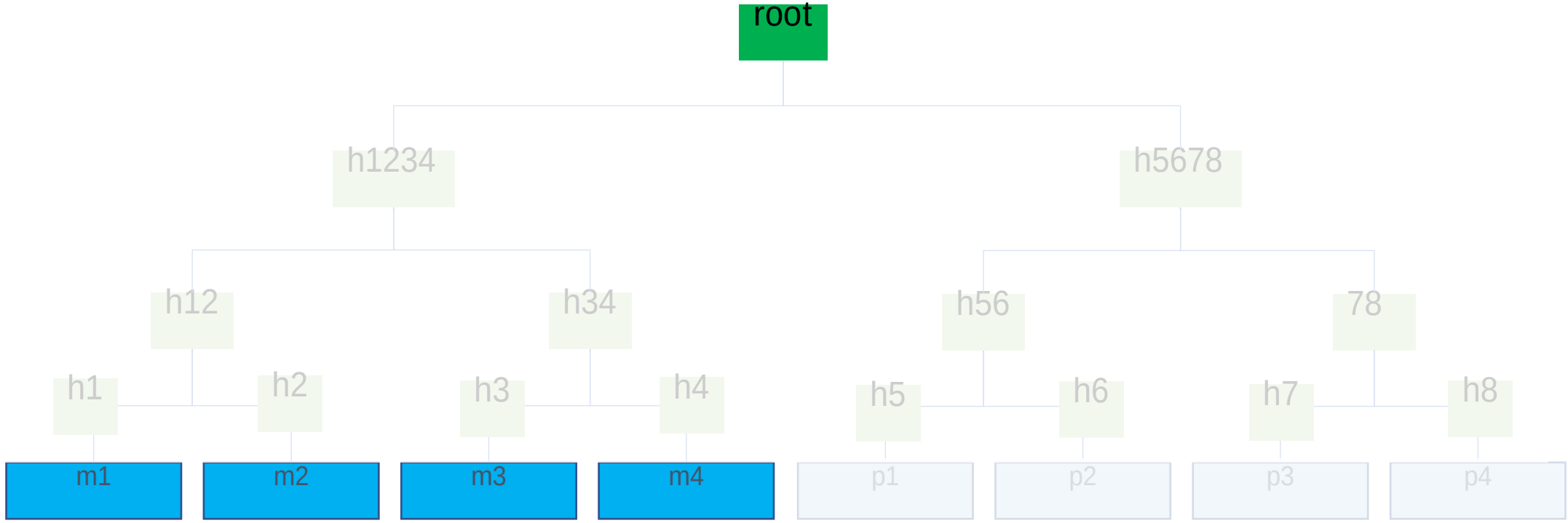
Introducing Erasure Coding



Any 50% symbols allow full recovery
(1-dimensional Reed-Solomon code, 1D-RS)

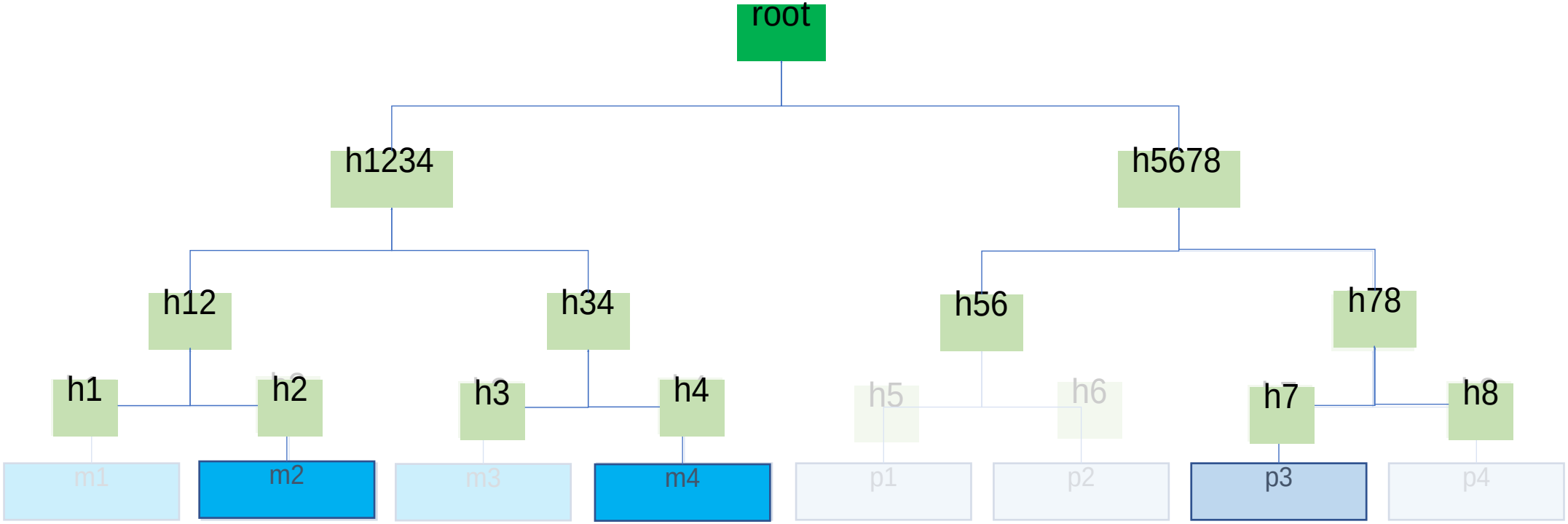
Merkle Tree of Erasure Coded Block

(Not coded Merkle tree!)



The miner only needs to publish the (uncoded) block and the root.
Any full node can reproduce the tree and verify.

Malicious Miner Can’t Hide



Show me m2

Show me m4

Show me p3

Show me …

The miner cannot disclose anymore!
It has to hide all the remaining 5.
This is easily detectable!

Coding v.s. no coding

No coding

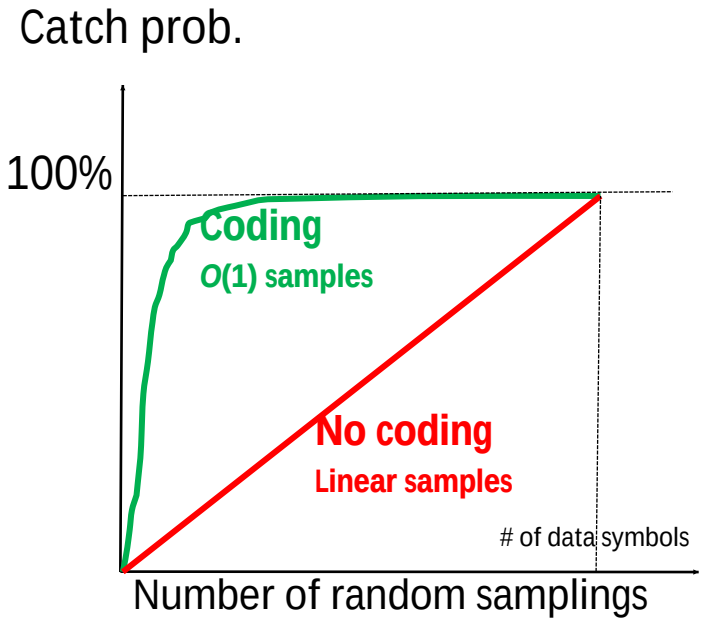
The malicious miner hides 1 data symbol

Probability of catching hiding after 2 random samplings?

Coding

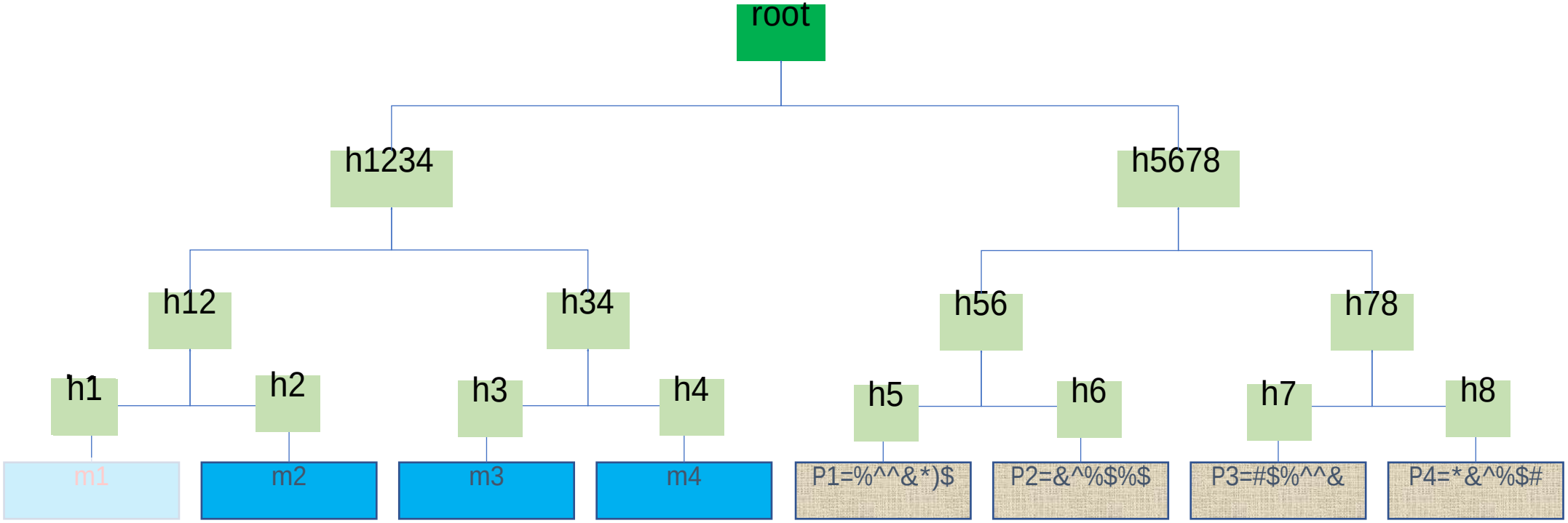
The malicious miner must hide 5 coded symbols

Probability of catching hiding after 2 random samplings?



But Malicious Miner Can Mess up with the Coding!

Introducing incorrect-coding proof



Incorrect-coding attack

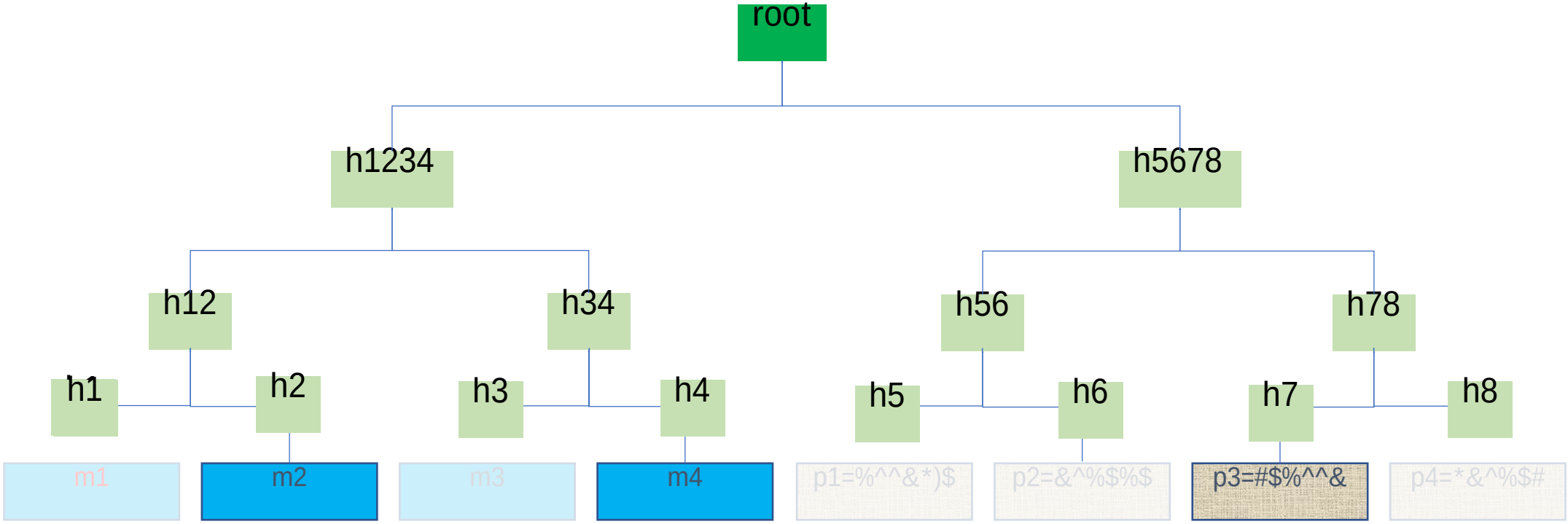
Miner can **encode incorrectly** and publish almost everything.
Light node can hardly catch hiding.

OK. Let me verify by reproducing.

(Proof size = block size !!!)

Incorrect-coding proof

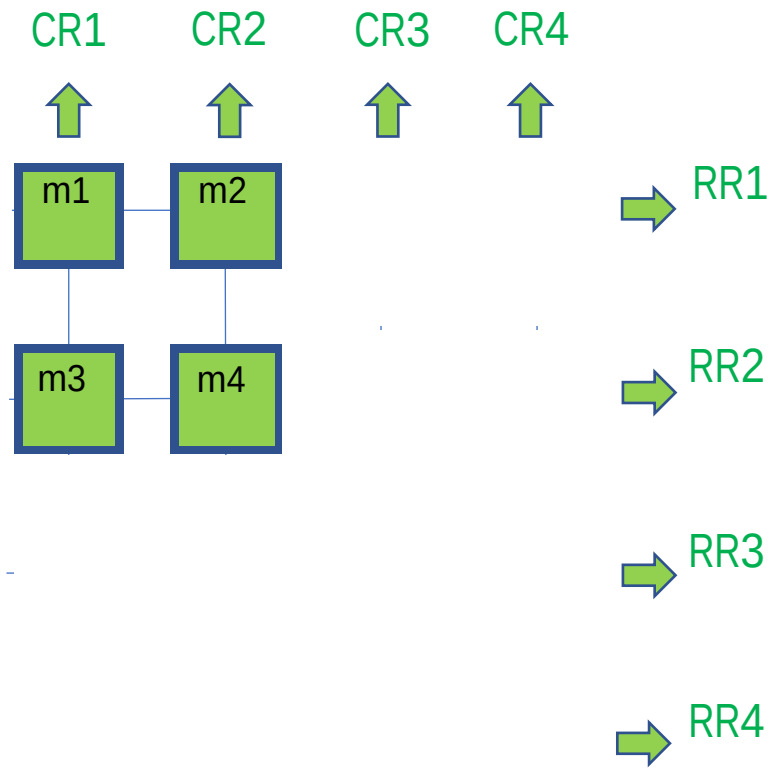
Summary of Costs



	# of roots	# of samples & Merkle proofs	Incorrect-coding proof size	En/decoding complexity
1D-RS	1	$O(1)$	$O(K)$	$O(K_2)$

K: block size in terms of number of symbols it is partitioned into.

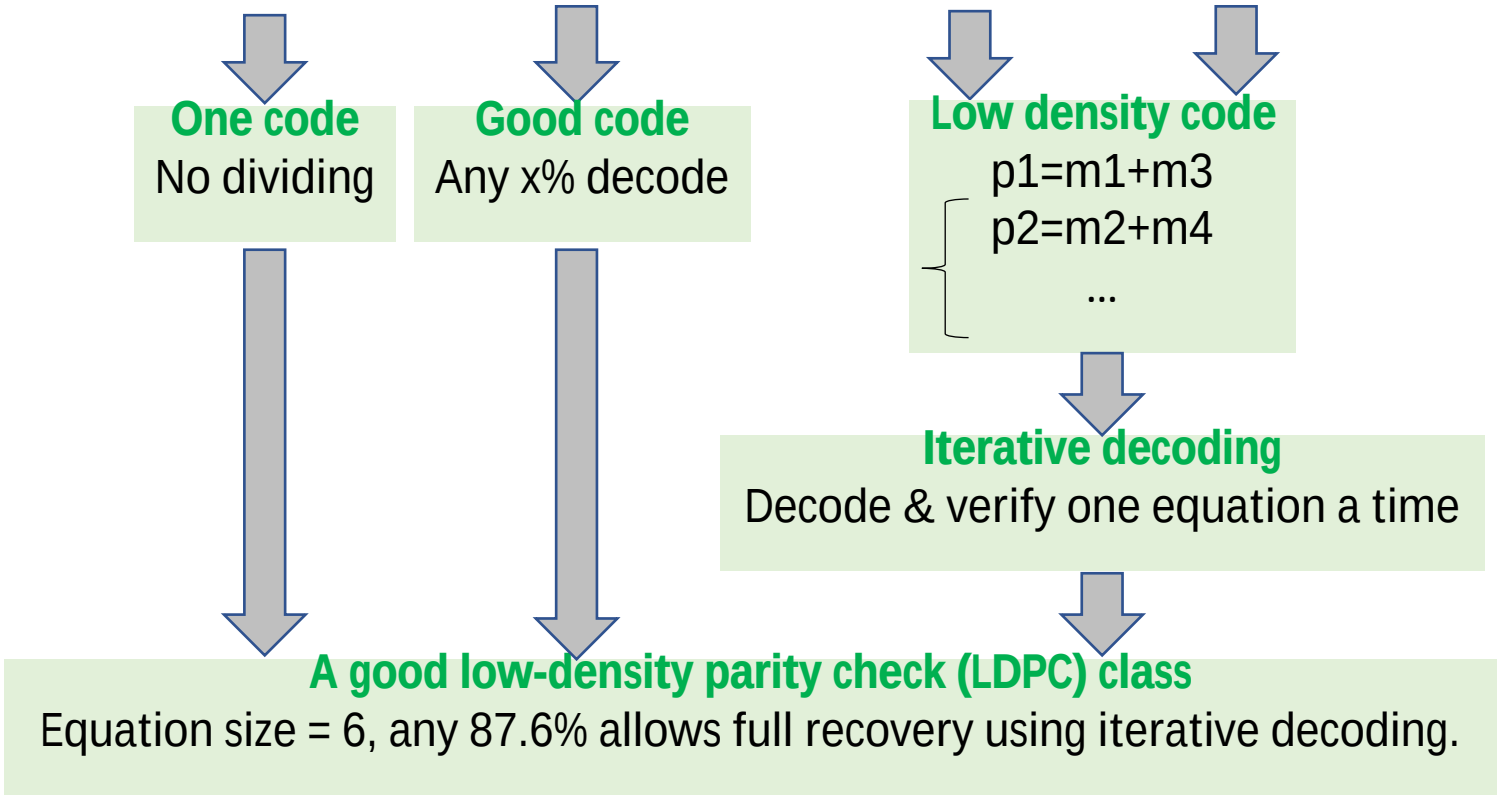
2D-RS: Divide and Conquer [1]



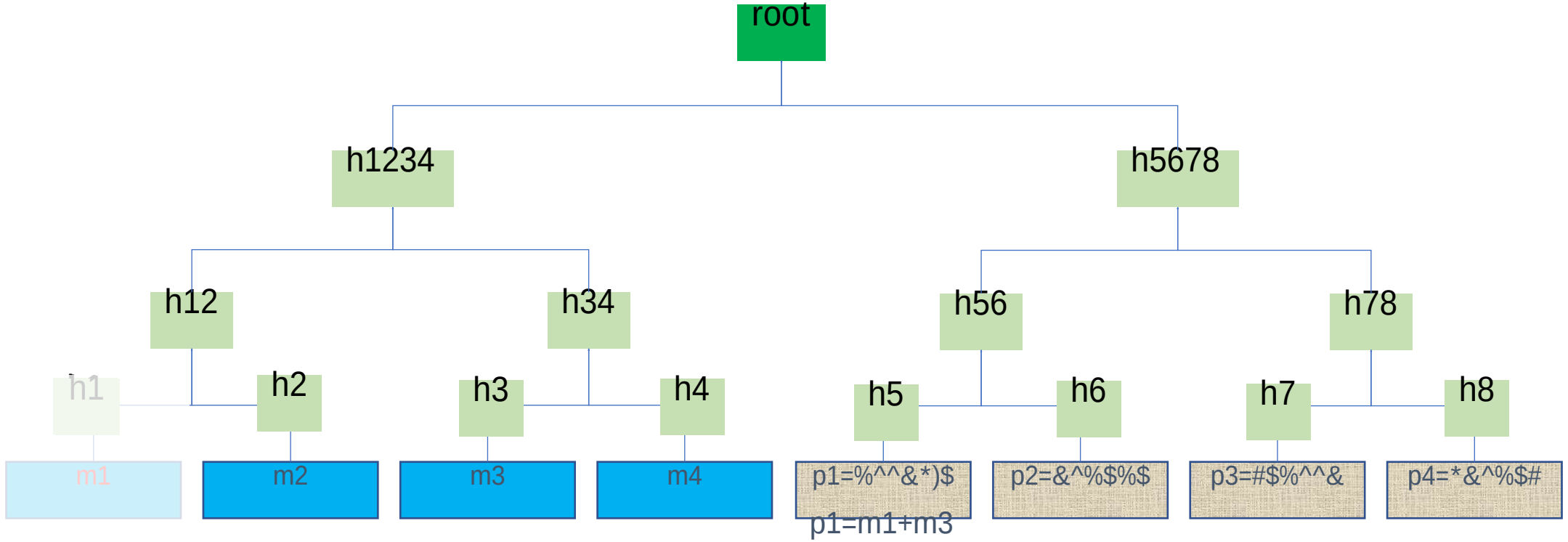
[1] Al-Bassam, M., Sonnino, A., Buterin, V., *Fraud and data availability proofs: Maximising light client security and scaling blockchains with dishonest majorities, 2018*

- Motivation: blockchain light node
- The data availability problem: definition and assumptions
- How could erasure coding help?
- **The optimal solution**
- Performance

The Derivation



Attack: Incorrect coding + hide hashes



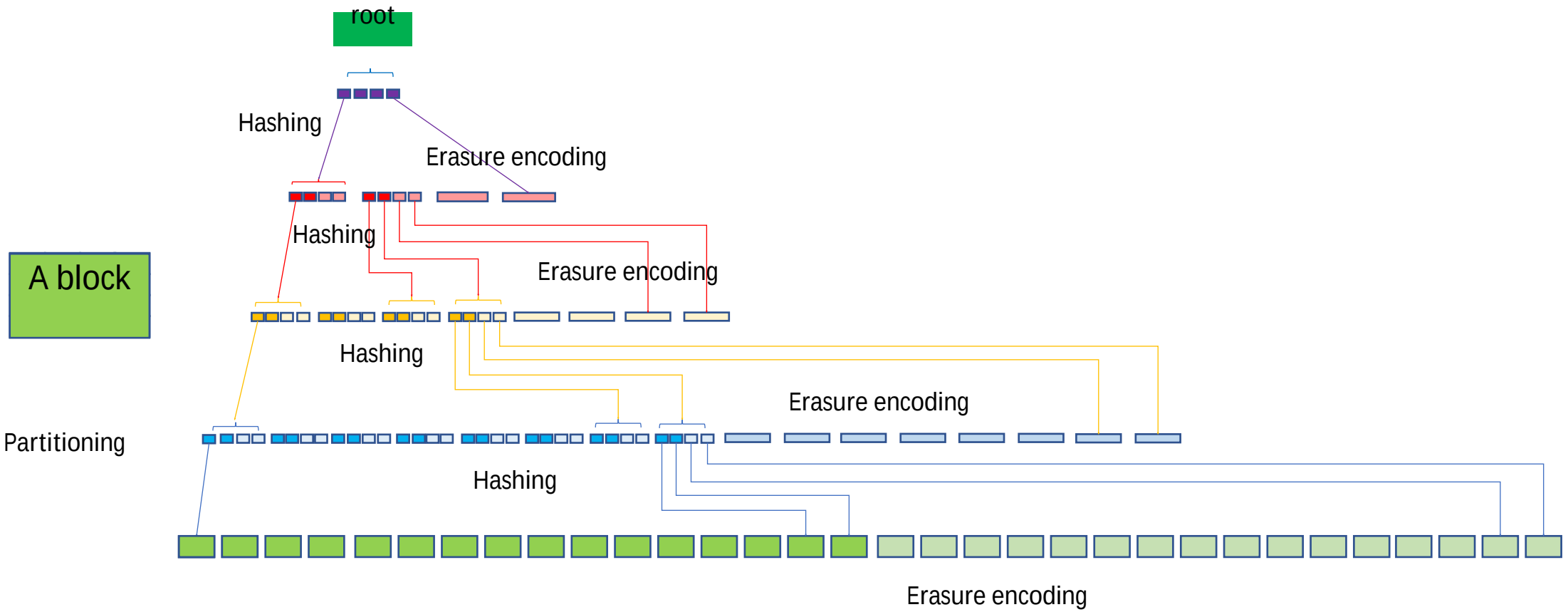
I have m3 & p1, let me recover m1 as m1=p1-m3

Is my m1 correct? Wait, I don't have h1 !

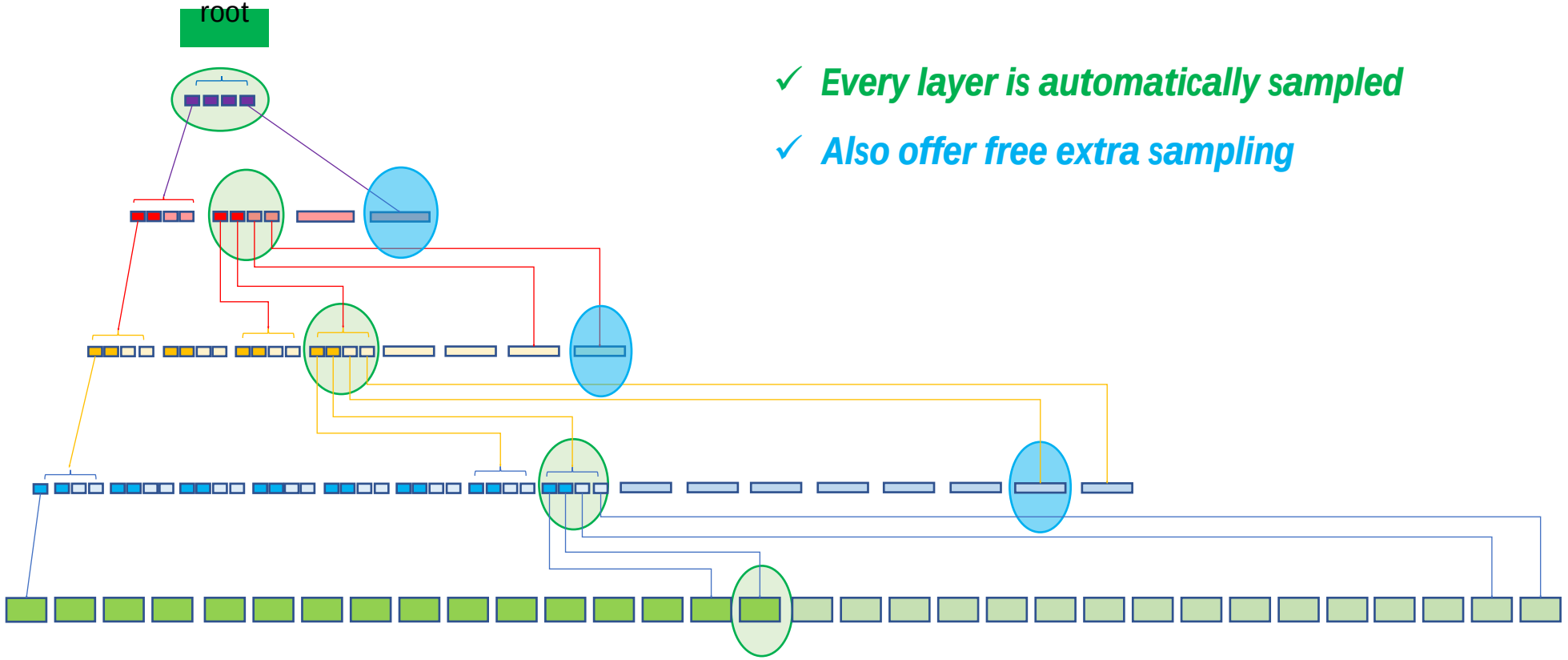
This idea only works if:
all the layer-1 hashes are available.

Err. I should also check layer-1 availability …

Introducing: Coded Merkle Tree

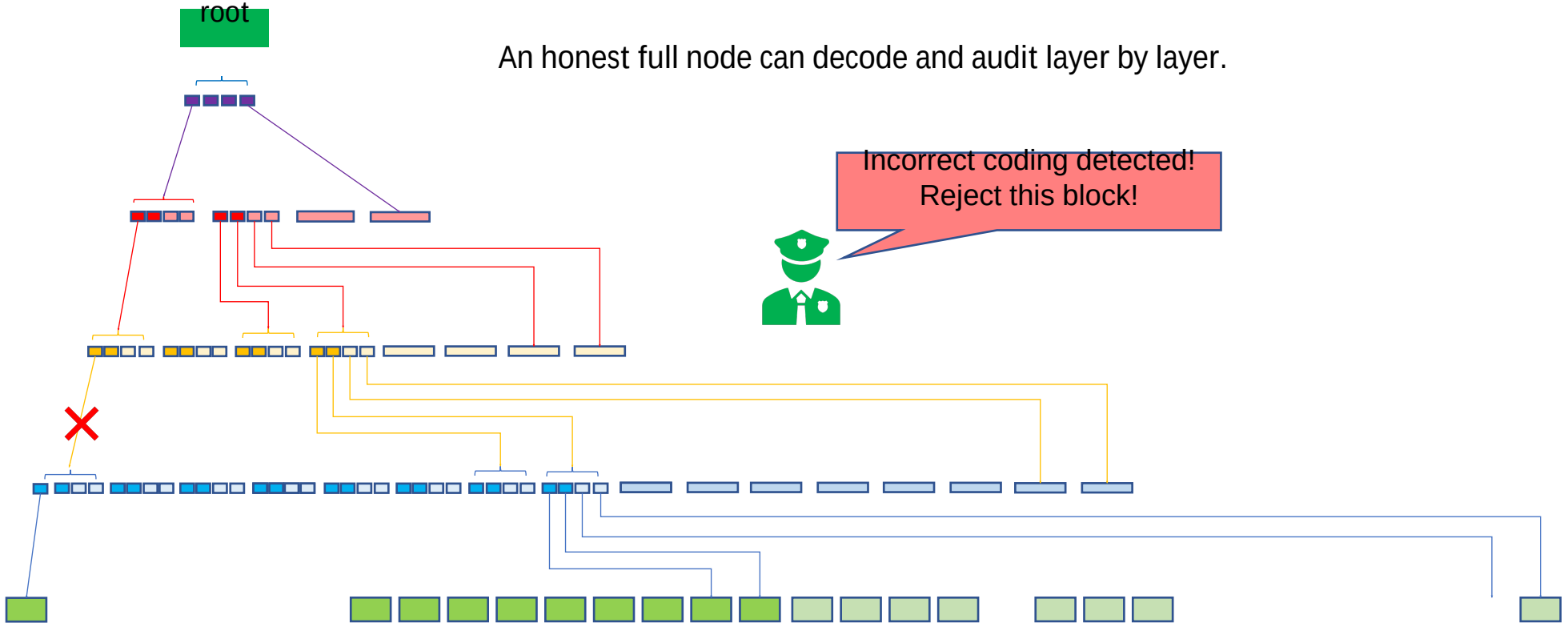


Light node sampling



- ✓ *Every layer is automatically sampled*
- ✓ *Also offer free extra sampling*

Honest Full Node Layer-by-Layer Recovery



- Motivation: blockchain light node
- The data availability problem: definition and assumptions
- How could erasure coding help?
- Our solution
- **Performance**

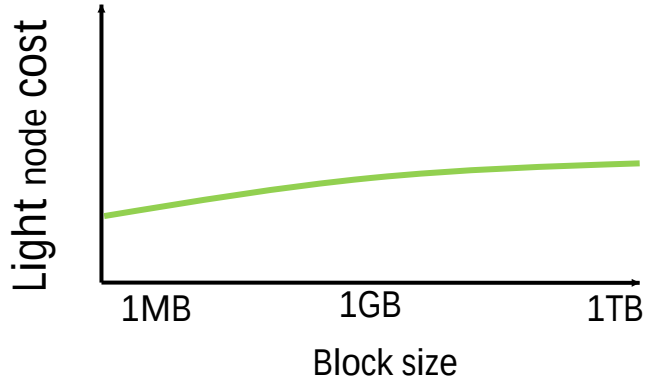
Performance

If K=65536, encode to 65536*4 coded symbols:

	# of roots	# of samples & Merkle proofs for 99% confidence	Incorrect-coding proof size (# of symbols)	En/decoding complexity
1D-RS	1	7	65536	13B ops
2D-RS	1025	15	256	50M ops
CMT	1	32	6	1.2M ops

Summary

CMT Enable data availability check at almost constant costs



Extremely simple to implement. Just
add CMT root (32 Bytes) to block header or a smart contract.

Designed to scale blockchains, but can be used for any
decentralized systems vulnerable to data hiding.